MODULE *sync_scheduler*

Zcash block-download scheduler specification.

Models the inventory-routing registry and download pipeline that sit one layer above the per-connection protocol in *protocol.tla*. Where *protocol.tla* models a single connection pulling blocks from one partner, this module models the node choosing which of many peers to ask for each block — the layer where Zebra's genesis-to-tip sync stall lived (*ZcashFoundation*/zebra $\neq$ 10679, symptom $\neq$ 5709).

A local syncer downloads blocks 1 .. *MaxBlock* from a pool of peers. Each peer genuinely holds some blocks (Holders); requesting a block a peer does not have yields an explicit notfound, while any request may also fail transiently (timeout or dropped connection). The scheduler keeps an availability registry and only routes a getdata to a peer not already marked missing for that block.

Two booleans select Zebra's pre- and post-fix behaviour:

BuggyRouting = TRUE → a transient error marks the peer missing for the block, poisoning the routing registry.
= FALSE → a transient error leaves the entry unknown.

BuggyDrop = TRUE → a notfound silently drops the block from the work set, wedging the frontier.
= FALSE → a notfound re-queues the block (bounded retries), routed away from the notfound peer.

The safety invariant *RegistryHonest* and the liveness property *EventuallyAllVerified* both hold under the fixed behaviour and are violated under the buggy behaviour, reproducing the stall as a *TLC* counterexample.

See documents/sync-stall-*modeling.md* for the full write-up.

EXTENDS *Naturals*, *FiniteSets*

CONSTANT *Peers* — set of peer ids
CONSTANT *MaxBlock* — blocks to download are 1 .. *MaxBlock*
CONSTANT *LaggingPeers* — peers that are behind: they hold only blocks 1 .. *LagTip*
CONSTANT *LagTip* — chain height of a lagging peer (< *MaxBlock*)
CONSTANT *MaxRetries* — bound on re-queue attempts per block after a notfound
CONSTANT *BuggyRouting* — TRUE reproduces the registry-poisoning bug
CONSTANT *BuggyDrop* — TRUE reproduces the silent-drop bug

$Blocks \triangleq 1 \,..\, MaxBlock$
$NONE \triangleq$ "none" — sentinel for "no peer"; must not be a member of *Peers*

Ground truth: a lagging peer holds blocks up to *LagTip*; every other peer is at tip and holds them all. The scheduler never reads this directly — it learns a peer's contents only through deliver/notfound. A degraded peer set (most peers lagging) is what made the stall visible in practice.

$PeerTip(p) \triangleq$ IF $p \in LaggingPeers$ THEN $LagTip$ ELSE $MaxBlock$
$Holds(p, b) \triangleq b \leq PeerTip(p)$

VARIABLES
verified, — subset of *Blocks* downloaded and verified (the frontier)
pending, — subset of *Blocks* the scheduler still wants and may request

inflight,	[$Blocks \to Peers \cup \{NONE\}$] peer a block is requested from
avail,	[$Peers \to$ [$Blocks \to \{$"has", "missing", "unknown"$\}$]] registry
retries	[$Blocks \to 0 \,..\, MaxRetries$] re-queue attempts used

$vars \triangleq \langle verified,\ pending,\ inflight,\ avail,\ retries \rangle$

$Init \triangleq$
$\quad \land\ verified = \{\}$
$\quad \land\ pending = Blocks$
$\quad \land\ inflight = [b \in Blocks \mapsto NONE]$
$\quad \land\ avail = [p \in Peers \mapsto [b \in Blocks \mapsto$ "unknown"$]]$
$\quad \land\ retries = [b \in Blocks \mapsto 0]$

The scheduler routes a getdata for a pending block to a peer not already marked missing for it (peer-to-candidate choice over the un-poisoned peers).

$Request \triangleq$
$\quad \exists\, b \in pending :$
$\quad\quad \exists\, p \in Peers :$
$\quad\quad\quad \land\ inflight[b] = NONE$
$\quad\quad\quad \land\ avail[p][b] \neq$ "missing"
$\quad\quad\quad \land\ inflight' = [inflight \text{ EXCEPT } ![b] = p]$
$\quad\quad\quad \land\ pending' = pending \setminus \{b\}$
$\quad\quad\quad \land$ UNCHANGED $\langle verified,\ avail,\ retries \rangle$

The in-flight peer genuinely has the block: it is delivered and verified, and the registry correctly records that the peer has it.

$DeliverBlock \triangleq$
$\quad \exists\, b \in Blocks :$
$\quad\quad \land\ inflight[b] \neq NONE$
$\quad\quad \land\ Holds(inflight[b],\ b)$
$\quad\quad \land\ verified' = verified \cup \{b\}$
$\quad\quad \land\ avail' = [avail \text{ EXCEPT } ![inflight[b]][b] =$ "has"$]$
$\quad\quad \land\ inflight' = [inflight \text{ EXCEPT } ![b] = NONE]$
$\quad\quad \land$ UNCHANGED $\langle pending,\ retries \rangle$

The in-flight peer genuinely lacks the block and says so. Marking the peer missing for this block is correct — an explicit notfound. What the scheduler does with the block then is the *BuggyDrop* switch: drop it, or re-queue it (under the retry bound) so it is routed to a different peer next time.

$NotFound \triangleq$
$\quad \exists\, b \in Blocks :$
$\quad\quad$ LET $p \triangleq inflight[b]$ IN
$\quad\quad \land\ p \neq NONE$
$\quad\quad \land\ \neg Holds(p,\ b)$
$\quad\quad \land\ inflight' = [inflight \text{ EXCEPT } ![b] = NONE]$

$\land\ avail' \quad = [avail \text{ EXCEPT } ![p][b] = \text{“missing”}]$
$\land$ IF $\neg BuggyDrop \land retries[b] < MaxRetries$
THEN $\land\ pending' = pending \cup \{b\}$
$\land\ retries' \ = [retries \text{ EXCEPT } ![b] = @ + 1]$
ELSE $\land$ UNCHANGED $pending$ silent drop (buggy), or retries exhausted
$\land$ UNCHANGED $retries$
$\land$ UNCHANGED $verified$

Any in-flight request may fail transiently (timeout or dropped connection), independent of whether the peer has the block. The request is abandoned and the block re-queued; the *BuggyRouting* switch decides the registry entry — poisoned (missing) or left unknown.

$TransientError \triangleq$
$\exists\, b \in Blocks :$
LET $p \triangleq inflight[b]$ IN
$\land\ p \neq NONE$
$\land\ inflight' = [inflight \text{ EXCEPT } ![b] = NONE]$
$\land\ pending' = pending \cup \{b\}$
$\land\ avail' \quad = [avail \text{ EXCEPT } ![p][b] = \text{IF } BuggyRouting \text{ THEN “missing” ELSE “unknown”}]$
$\land$ UNCHANGED $\langle verified, retries \rangle$

$Next \triangleq$
$\lor\ Request$
$\lor\ DeliverBlock$
$\lor\ NotFound$
$\lor\ TransientError$

Strong fairness on the progress actions encodes the real-world assumption that the node keeps trying and a willing peer eventually responds. The transient error gets no fairness — it is adversarial but never forced.

$Spec \triangleq$
$Init$
$\land\ \Box[Next]_{vars}$
$\land\ \mathrm{SF}_{vars}(Request)$
$\land\ \mathrm{SF}_{vars}(DeliverBlock)$
$\land\ \mathrm{SF}_{vars}(NotFound)$

Liveness: every block is eventually downloaded and verified. The genesis-to-tip stall is a violation of this property.

$EventuallyAllVerified \triangleq \Diamond(verified = Blocks)$

Safety invariants.

$$
\begin{array}{l}
\mathit{TypeOK} \triangleq \\
\quad \land \quad \mathit{verified} \subseteq \mathit{Blocks} \\
\quad \land \quad \mathit{pending} \subseteq \mathit{Blocks} \\
\quad \land \quad \mathit{inflight} \in [\mathit{Blocks} \to \mathit{Peers} \cup \{\mathit{NONE}\}] \\
\quad \land \quad \mathit{avail} \quad \in [\mathit{Peers} \to [\mathit{Blocks} \to \{\text{“has”},\ \text{“missing”},\ \text{“unknown”}\}]] \\
\quad \land \quad \mathit{retries} \ \in [\mathit{Blocks} \to 0 \mathinner{..} \mathit{MaxRetries}]
\end{array}
$$

The registry never marks a peer missing for a block it actually holds. This is the formal statement of “a timeout is not a notfound”: under *BuggyRouting* a transient error on a holder violates it immediately.

$$
\begin{array}{l}
\mathit{RegistryHonest} \triangleq \\
\quad \forall\, p \in \mathit{Peers} : \\
\qquad \forall\, b \in \mathit{Blocks} : \\
\qquad\quad \mathit{avail}[p][b] = \text{“missing”} \Rightarrow \neg \mathit{Holds}(p,\ b)
\end{array}
$$

A peer is only ever recorded as having a block it genuinely holds.

$$
\begin{array}{l}
\mathit{RegistryHasIsSound} \triangleq \\
\quad \forall\, p \in \mathit{Peers} : \\
\qquad \forall\, b \in \mathit{Blocks} : \\
\qquad\quad \mathit{avail}[p][b] = \text{“has”} \Rightarrow \mathit{Holds}(p,\ b)
\end{array}
$$

A verified block was genuinely available from some peer.

$$
\begin{array}{l}
\mathit{VerifiedAreReal} \triangleq \\
\quad \forall\, b \in \mathit{verified} : \\
\qquad \exists\, p \in \mathit{Peers} : \mathit{Holds}(p,\ b)
\end{array}
$$